

# AUTOM-03: Monaco Configuration-as-Code

**Series:** AUTOM — Dynatrace Automation | **Notebook:** 3 of 9 | **Created:** January 2026 | **Last Updated:** 09/28/2026

Monaco (Monitoring as Code) is Dynatrace's official CLI tool for configuration management. It uses YAML files to define configurations and supports version control, CI/CD integration, and multi-environment deployments.

## Table of Contents

- 1. [Introduction](#)
- 2. [Getting Started](#)
- 3. [Project Structure](#)
- 4. [Configuration Files](#)
- 5. [Common Operations](#)
- 6. [Advanced Features](#)
- 7. [Next Steps](#)

## Prerequisites

Before starting this notebook, ensure you have:

| Requirement | Description                                                                         |
|-------------|-------------------------------------------------------------------------------------|
| Monaco CLI  | Installed from the GitHub releases binary (Dynatrace publishes no Homebrew formula) |

| Requirement                    | Description                                                                                                                                                                                            |
|--------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Platform token or OAuth client | For platform configuration — SLOs ( <code>slo:slos:read</code> / <code>slo:slos:write</code> ), workflows, documents, segments, buckets                                                                |
| API Token                      | Token with <code>settings.read</code> , <code>settings.write</code> (Settings 2.0), plus <code>ReadConfig</code> , <code>WriteConfig</code> only if you still manage classic Configuration API objects |
| Tenant URL                     | Your Dynatrace SaaS tenant URL                                                                                                                                                                         |

## Learning Objectives

By the end of this notebook, you will:

- Understand Monaco's configuration-as-code model
- Know how to download existing configurations
- Be able to create and deploy configurations via YAML
- Handle multi-environment deployments

## 1. Introduction

### Why Monaco?

| Benefit           | Description                                 |
|-------------------|---------------------------------------------|
| Version Control   | Track all changes in Git                    |
| Repeatability     | Deploy same config to multiple environments |
| Review Process    | Use pull requests for config changes        |
| Disaster Recovery | Quickly restore configurations              |
| Documentation     | YAML files serve as living documentation    |

## Monaco vs Direct API

| Aspect           | Monaco                             | Settings API     |
|------------------|------------------------------------|------------------|
| Format           | YAML files                         | JSON payloads    |
| State management | Implicit (by name)                 | Manual tracking  |
| Multi-tenant     | Built-in support                   | Custom scripting |
| Dry run          | Yes ( <code>--dry-run</code> )     | No               |
| Delete orphans   | Yes ( <code>delete</code> command) | Manual           |

**Already a Terraform shop?** See **AUTOM-01 §5: When a Terraform Shop Should Add Monaco** for the four patterns where Monaco still earns its place — most commonly bulk download from an existing tenant, multi-tenant identical-config deployment, and app-team self-service without state management.

## 2. Getting Started

### Installation

Monaco ships as a single binary on GitHub releases; there is no Homebrew formula. Pick the asset for your OS and architecture (the install guide also lists `monaco-darwin-amd64` , `monaco-linux-arm64` , `monaco-linux-386` and a SHA-256 file for each):

#### macOS (Apple silicon):

```
curl -L https://github.com/Dynatrace/dynatrace-configuration-as-code/releases/latest/download/monaco-darwin-arm64 -o monaco
chmod +x monaco
```

#### Linux (x86-64):

```
curl -L https://github.com/Dynatrace/dynatrace-configuration-as-code/releases/latest/download/monaco-linux-amd64 -o monaco
chmod +x monaco
```

**Windows:** download the `.exe` asset from the same releases page.

**Verify installation:**

```
monaco version
```

## Environment Setup

Create environment variables for authentication:

```
export DT_TENANT_URL="https://{tenant-id}.live.dynatrace.com"
export DT_API_TOKEN="<your-api-token>"
export DT_PLATFORM_TOKEN="<your-platform-token>" # SLOs, workflows,
documents, segments, buckets
```

Monaco never takes a secret on the command line. Both the manifest and the `--token` download flag take the **name** of the environment variable that holds the token (`DT_API_TOKEN`), not its value.

---

## Your First Download

Download all configurations from your tenant:

```
monaco download \
  --url "$DT_TENANT_URL" \
  --token DT_API_TOKEN \
  --output-folder ./downloaded-config
```

`--token` names the environment variable (no `$`). An access token covers settings and classic configuration APIs; to also download platform configurations (workflows, documents, buckets, segments), add `--platform-token <VAR_NAME>` or the `--oauth-client-id` / `--oauth-client-secret` pair — access tokens and platform tokens are not interchangeable.

Download only the platform configuration types — SLOs, segments, workflows and documents. The `--only-*` flags can be combined (Monaco 2.23.1+):

```
monaco download \  
  --url "$DT_TENANT_URL" \  
  --token DT_API_TOKEN \  
  --platform-token DT_PLATFORM_TOKEN \  
  --output-folder ./downloaded-config \  
  --only-slo-v2 --only-segments --only-automation --only-documents
```

The full set of `--only-*` flags is `--only-settings`, `--only-apis`, `--only-slo-v2`, `--only-segments`, `--only-automation`, `--only-documents`, `--only-buckets` and `--only-openpipeline` (read from the download command's source, `cmd/monaco/download/download_command.go`).

**Classic — inventorying what you still have before the upgrade.** Download specific Settings 2.0 schemas with `--settings-schema` (`--api` is for **classic** Configuration APIs only):

```
monaco download \  
  --url "$DT_TENANT_URL" \  
  --token DT_API_TOKEN \  
  --output-folder ./downloaded-config \  
  --settings-schema builtin:management-zones,builtin:tags.auto-tagging
```

**Downloading classic schemas is the point here, not an oversight.** Both schemas above are marked **Blocked at upgrade** in AUTOM-02's Settings 2.0 catalog — they work today and stop answering once your tenant moves to the latest Dynatrace. That makes `monaco download` the fastest way to *inventory* what you still have before migrating it: every object, in version control, in one command. Keep the export as your migration worksheet; retarget the deployment. What it should not become is a template for new configuration.

## 3. Project Structure

---

### Recommended Layout

```
dynatrace-config/
├─ manifest.yaml          # Project manifest
├─ environments/
│   ├─ development.yaml  # Dev environment config
│   ├─ staging.yaml       # Staging config
│   └─ production.yaml   # Production config
└─ projects/
    └─ my-project/
        ├─ slo-v2/        # service-level objectives (modern SLO app)
        │   └─ config.yaml
        ├─ segments/
        │   └─ config.yaml
        ├─ workflows/     # problem routing and other automations
        │   └─ config.yaml
        └─ anomaly-detectors/ # builtin:davis.anomaly-detectors (Settings
2.0)
            └─ config.yaml
```

Classic estates add `management-zones/`, `auto-tagging/` and `alerting-profiles/` here until the upgrade — see the *Classic* blocks in §4.

### Manifest File

The `manifest.yaml` defines your project:

```

manifestVersion: 1.0

projects:
  - name: my-project
    path: projects/my-project

environmentGroups:
  - name: default
    environments:
      - name: development
        url:
          type: environment
          value: DT_DEV_URL
        auth:
          token:
            name: DT_DEV_TOKEN
          platformToken:
            name: DT_DEV_PLATFORM_TOKEN
      - name: production
        url:
          type: environment
          value: DT_PROD_URL
        auth:
          token:
            name: DT_PROD_TOKEN
          platformToken:
            name: DT_PROD_PLATFORM_TOKEN

```

`url` accepts `type: environment` + `value: <VAR_NAME>`, but every `auth` entry takes only `name: <VAR_NAME>` — Monaco always loads secrets from environment variables. `platformToken` (Monaco 2.24.0+) — or an `oAuth` client in its place — is what lets the project manage SLOs, workflows, documents, buckets and segments; drop it only for a project that manages Settings 2.0 and classic configuration alone.

---

## 4. Configuration Files

---

### Basic Configuration Structure

Every config is an `id`, a `config:` block (name, template, parameters) and a `type:`. For a Settings 2.0 object the type names a schema; for a platform object it names the platform type:

```

configs:
  - id: checkout-availability
    config:
      name: Checkout availability
      template: slo.json
    type: slo-v2

```

## SLO Example ( type: slo-v2 )

slo-v2/config.yaml:

```

configs:
  - id: checkout-availability
    config:
      name: Checkout availability
      template: checkout-availability.json
    type: slo-v2

```

slo-v2/checkout-availability.json:

```

{
  "name": "{{ .name }}",
  "description": "Request success ratio, rolling 30 days",
  "tags": ["app:checkout", "owner:payments"],
  "criteria": [
    {
      "target": 99.5,
      "warning": 99.9,
      "timeframeFrom": "now-30d",
      "timeframeTo": "now"
    }
  ],
  "customSli": {
    "indicator": "timeseries { total = sum(dt.service.request.count), failures = sum(dt.service.request.failure_count) }\n| fieldsAdd sli = ((total[] - failures[]) / total[]) * 100\n| fieldsRemove total, failures"
  }
}

```

The payload is the SLO Service API's own shape — `criteria` is a **list**, and the SLI is a DQL `indicator` producing a timeseries field named `sli` (SLO-02's availability query, minus `from:` / `interval:`, which the criteria supply). It matches Monaco's own `slo-v2` integration-test template field for field. `type: slo-v2` needs Monaco **v2.22.0+** and platform auth, and despite the name it is the **modern** SLO app — the classic SLO is a

different Monaco type, and the Terraform resource `dynatrace_slo_v2` is classic too (SLO-05).

## Other Platform Types

| Type block                                                                                                                             | Manages                                                                                                      | Since                          |
|----------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------|--------------------------------|
| <code>type: slo-v2</code>                                                                                                              | SLOs on the modern SLO app                                                                                   | v2.22.0                        |
| <code>type: segment</code>                                                                                                             | Segments (the filtering successor to management zones)                                                       | v2.19.0                        |
| <code>type: {automation: {resource: workflow}}</code> — also <code>business-calendar</code> , <code>scheduling-rule</code>             | Workflows (the routing successor to alerting profiles + problem notifications)                               | v2.6.0                         |
| <code>type: {document: {kind: dashboard}}</code> — also <code>notebook</code> , <code>launchpad</code> ; optional <code>private</code> | Dashboards, notebooks, launchpads                                                                            | v2.15.0<br>(launchpad v2.18.0) |
| <code>type: bucket</code>                                                                                                              | Grail buckets                                                                                                | v2.9.0                         |
| <code>type: {settings: {schema: builtin:davis.anomaly-detectors, scope: environment}}</code>                                           | Davis anomaly detectors (the successor to custom metric events) — a Settings 2.0 schema that carries forward | —                              |

**Download a workflow or segment before you author one.** A workflow template is the full Automation API payload (trigger, tasks, positions) and a segment template is the segment editor's JSON filter tree — neither is sensible to hand-write. Build one in the app, `monaco download --only-automation` / `--only-segments`, and parameterize the result.

**Sources:** [Monaco YAML configuration — type fields \(DT docs\)](#) — the type blocks and version gates in the table; [slo-v2 test template \(Dynatrace GitHub\)](#) — payload shape, read 09/28/2026; [Monaco v2.22.0 release notes \(Dynatrace GitHub\)](#) — "Support for service-level objectives leveraging Grail".

## Classic Configuration — existing estates on unupgraded tenants

**Dynatrace Classic — maintain, don't author.** Management zones, auto-tagging and alerting profiles are the configurations most existing Monaco projects contain, so their shape is kept below. All three are marked **Blocked at upgrade** in AUTOM-02's Settings 2.0 catalog and are on Dynatrace's published [Settings 2.0 schemas that are removed in Latest Dynatrace \(DT docs\)](#) — *"None of the schemas on this page are visible in Latest Dynatrace."*

That matters more for Monaco than for the raw API. Monaco resolves a schema before writing objects, so once a schema is removed, `monaco deploy` fails for every config that references it — and it fails at deploy time, in your pipeline, not at edit time. Successors: management zone → `segment` (filtering) + IAM policies (access); alerting profile → `automation workflow`; auto-tagging → primary fields/tags set at ingest (FAQ-02).

### Management Zone Example (classic)

**config.yaml:**

```
configs:
  - id: production-mz
    config:
      name: Production
      template: production-mz.json
    type:
      settings:
        schema: builtin:management-zones
        scope: environment

  - id: staging-mz
    config:
      name: Staging
      template: staging-mz.json
    type:
      settings:
        schema: builtin:management-zones
        scope: environment
```

**production-mz.json:**

```
{
  "name": "{{ .name }}",
  "rules": [
    {
      "type": "SERVICE",
      "enabled": true,
      "conditions": [
        {
          "key": {
            "type": "STATIC",
            "attribute": "SERVICE_TAGS"
          },
          "comparisonInfo": {
            "type": "TAG",
            "operator": "EQUALS",
            "value": {
              "context": "CONTEXTLESS",
              "key": "environment",
              "value": "production"
            },
            "negate": false
          }
        }
      ]
    }
  ]
}
```

---

## Auto-Tagging Example (classic)

**config.yaml:**

```
configs:
- id: application-tag
  config:
    name: Application
    template: application-tag.json
  type:
    settings:
      schema: builtin:tags.auto-tagging
      scope: environment
```

**application-tag.json:**

```
{
  "name": "{{ .name }}",
  "rules": [
    {
      "type": "SERVICE",
      "enabled": true,
      "valueFormat": "{Service:DetectedName}",
      "propagationTypes": ["SERVICE_TO_PROCESS_GROUP_LIKE"],
      "conditions": []
    }
  ]
}
```

## Using Variables

Monaco v2 reads environment variables through **environment parameters**, declared under `parameters:` with a `name` (the variable) and an optional `default`. The template then refers to the parameter by its key — there is no `{{ .Env.X }}` syntax in v2:

**config.yaml:**

```
configs:
- id: checkout-availability
  config:
    name: Checkout availability
    template: checkout-availability.json
    parameters:
      owner_team:
        type: environment
        name: OWNER_TEAM
      slo_target:
        type: environment
        name: SLO_TARGET
        default: "99.5"
    type: slo-v2
```

In `checkout-availability.json`, use `"tags": ["owner:{{ .owner_team }}" ]` and `"target": {{ .slo_target }}` — unquoted, so the rendered JSON carries a number. Without a `default`, the deployment fails if the variable is unset.

---

## 5. Common Operations

---

### Deploy Configurations

Deploy to all environments:

```
monaco deploy manifest.yaml
```

Deploy to specific environment:

```
monaco deploy manifest.yaml --environment production
```

Dry run (check file structure without applying):

```
monaco deploy manifest.yaml --dry-run
```

### Validate Configurations

Monaco does **not** ship a standalone `monaco validate` subcommand. Validation runs as part of `monaco deploy --dry-run`, which loads the manifest, parses every project, checks that templates are valid JSON, and resolves cross-references — **without contacting the tenant**. It cannot validate the JSON payload against the schema, and it does not check reachability or token scopes: a deploy can still fail with HTTP 400 after a clean dry-run.

```
monaco deploy manifest.yaml --dry-run
```

### Delete Configurations

Remove configurations not in your project (orphans). The manifest is passed via `--manifest`, not as a positional argument:

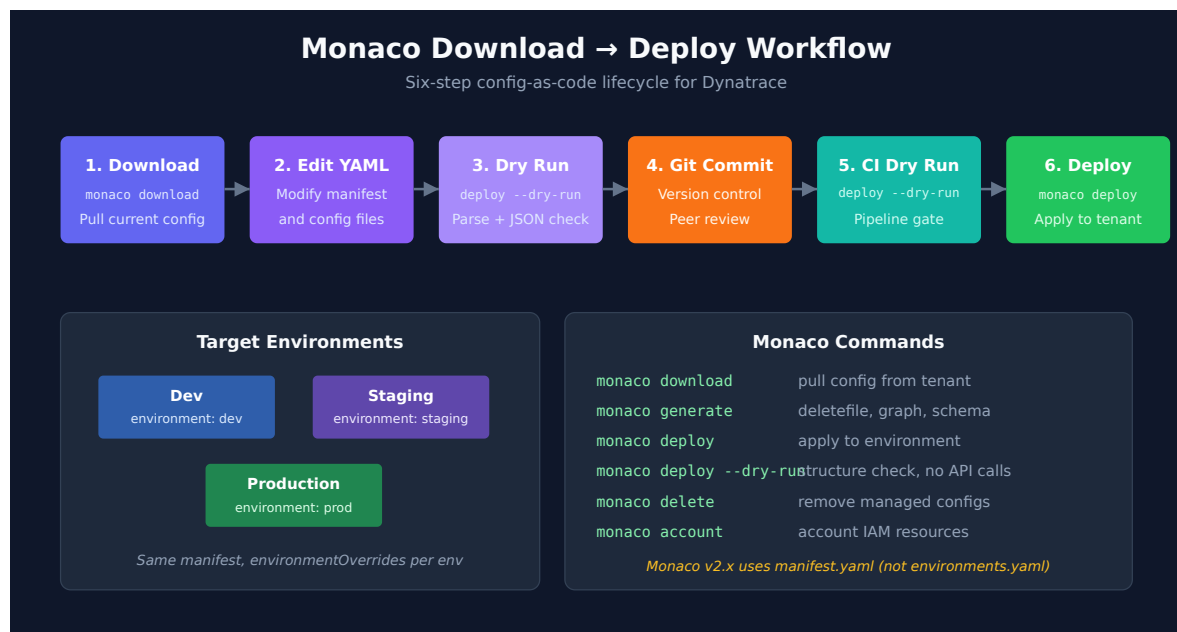
```
monaco delete --manifest manifest.yaml --file delete.yaml
```

**delete.yaml:**

```
delete:
  - project: my-project
    type: slo-v2
    id: checkout-availability
  # Classic, until the upgrade – a Settings 2.0 entry takes the schema as its
  type
  - project: my-project
    type: builtin:management-zones
    id: old-management-zone-id
```

Every non-classic entry needs **project** and **id** together (the Monaco config coordinate) or an **objectId** on its own; Monaco's delete-file loader rejects an entry with **id** and no **project**. The **type** is the config type ( **slo-v2** , **segment** , **workflow** , **document** , **bucket** ) or, for Settings 2.0, the schema ID.

## Download vs Deploy Workflow



## 6. Advanced Features

### Configuration Dependencies

Reference another configuration with a **reference parameter** — `[project, configType, configId, property]`, or the long form with `type: reference`. The `configType` is the config type (`segment`, `slo-v2`, `workflow`, ...) or, for Settings 2.0, the schema ID; `property: id` resolves to the referenced object's real Dynatrace ID at deploy time, and Monaco orders the deployment so the referenced object exists first. Here an SLO is scoped by a segment deployed from the same project:

```
configs:
- id: checkout-availability
  config:
    name: Checkout availability
    template: checkout-availability.json
    parameters:
      checkout_segment_id:
        type: reference
        configType: segment
        configId: checkout-production
        property: id
    type: slo-v2
```

In the template, scope the SLI with `"filterSegments": [{"id": "${checkout_segment_id}"}]` inside `customSli`. Because the segment ID is resolved per environment, the same config deploys to every tenant without a hard-coded ID — the portability problem S2S-07 spends a section on.

**Classic equivalent:** an alerting profile referencing a management zone —

`management_zone_id: ["my-project", "builtin:management-zones", "production-mz", "id"]` on a `builtin:alerting.profile` config, used in the template as `${management_zone_id}`. Same mechanism, blocked schemas.

### Conditional Configurations

`skip` is a boolean on `config:`. To deploy a config to only some environments, set a default and flip it per environment (or per environment group with `groupOverrides`):

```

configs:
  - id: production-only-config
    config:
      name: Production Detector
      template: detector.json
      skip: true
    type:
      settings:
        schema: builtin:davis.anomaly-detectors
        scope: environment
    environmentOverrides:
      - environment: production
        override:
          skip: false

```

## Environment-Specific Overrides

`environmentOverrides` and `groupOverrides` sit at the config level, as siblings of `config:` — not inside it:

```

configs:
  - id: checkout-availability
    config:
      name: Checkout availability
      template: checkout-availability.json
      parameters:
        slo_target: 99.0
    type: slo-v2
    environmentOverrides:
      - environment: production
        override:
          parameters:
            slo_target: 99.5

```

---

## Grouping Configurations

Monaco v2 has no per-config `group:` key. Two different things are grouped, and neither is a config group:

- **Projects** group configurations. Put related configs in one project directory and deploy just that project with `--project`.

- **Environment groups** ( `environmentGroups` in the manifest) group *target environments*. `--group` deploys to every environment in the named group; it does not select configs.

```
# Deploy only the web-application project, to every environment in the
"production" environment group
monaco deploy manifest.yaml --project web-application --group production
```

## Best Practices

| Practice                               | Description                                                                                                                                                                                  |
|----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Use meaningful IDs                     | IDs should describe the config purpose                                                                                                                                                       |
| Modular structure                      | Separate configs by domain (alerting, monitoring, etc.)                                                                                                                                      |
| Environment variables                  | Never hardcode tokens or URLs                                                                                                                                                                |
| Version control                        | Commit all changes with meaningful messages                                                                                                                                                  |
| Dry run first                          | Run <code>monaco deploy --dry-run</code> before applying — it catches YAML/JSON structure errors, not payload errors                                                                         |
| Document parameters                    | Comment complex configurations                                                                                                                                                               |
| Check upgrade status before you invest | Before building a long-lived project around a schema, confirm it is not marked <b>Blocked at upgrade</b> in AUTOM-02's catalog — a blocked schema takes the whole Monaco config type with it |

## Common Issues and Solutions

| Issue                   | Cause        | Solution                         |
|-------------------------|--------------|----------------------------------|
| "Config already exists" | Duplicate ID | Use unique IDs or download first |

| Issue                | Cause                                                                                                 | Solution                                                                                                                                                        |
|----------------------|-------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| "Schema not found"   | Typo in schema ID — <b>or</b> the schema was removed when the tenant upgraded to the latest Dynatrace | Check the exact schema name in the API. If it was correct and worked previously, check the schema's upgrade status in AUTOM-02's catalog before assuming a typo |
| "Template not found" | Wrong path                                                                                            | Use relative path from config.yaml                                                                                                                              |
| "Validation failed"  | Invalid JSON                                                                                          | Check JSON syntax in template                                                                                                                                   |

## 7. Next Steps

### Monaco in a CI/CD Pipeline (the most common next step)

Monaco itself is a CLI — to make it part of a GitOps flow, wire it into your CI/CD platform of choice. **AUTOM-07: CI/CD Integration** has the recipe per platform:

| Platform            | AUTOM-07 section                                                                       |
|---------------------|----------------------------------------------------------------------------------------|
| GitHub Actions      | §3 GitHub Actions                                                                      |
| GitLab CI/CD        | §4 GitLab CI/CD                                                                        |
| Bitbucket Pipelines | §5.1 Bitbucket Pipelines — Monaco Deploy                                               |
| Atlassian Bamboo    | §6 Atlassian Bamboo — adapt the Plan Specs YAML to call <code>monaco deploy</code>     |
| Azure DevOps        | §7 Azure DevOps Pipelines — adapt the Pipeline YAML to call <code>monaco deploy</code> |

For the **full sequenced path** from zero to a working pipeline, see **AUTOM-01 §6 First-Time Setup Path — Path B (Monaco target)**.

## When to Consider Adding Terraform

| Scenario                                                                             | Why Terraform wins                                                                           |
|--------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|
| Cross-system orchestration (cloud + Dynatrace + Git in one apply)                    | Monaco is Dynatrace-only                                                                     |
| State management + drift detection on critical resources                             | Monaco has no state file; drift detection happens via plan-reapply rather than state diffing |
| Part of larger IaC stack                                                             | Existing Terraform workflows extend naturally to Dynatrace                                   |
| Lifecycle protections ( <code>prevent_destroy</code> , <code>ignore_changes</code> ) | Monaco has no per-resource lifecycle policy                                                  |

For the framing of when a Terraform shop should *also* adopt Monaco (the reverse direction), see **AUTOM-01 §5: When a Terraform Shop Should Add Monaco**.

## Continue the Series

| Next Notebook                         | Focus                                                             |
|---------------------------------------|-------------------------------------------------------------------|
| <b>AUTOM-04: Terraform Provider</b>   | Infrastructure-as-code approach to Dynatrace configuration        |
| <b>AUTOM-07: CI/CD Integration</b>    | GitOps patterns for both Monaco and Terraform                     |
| <b>AUTOM-08: Migration Automation</b> | Tenant-to-tenant migration including <code>monaco download</code> |

## Additional Resources

- [Monaco commands reference \(DT docs\)](#) — "A dry-run doesn't connect to Dynatrace and can't validate the content of the JSON sent to Dynatrace."
- [Monaco configuration YAML reference \(DT docs\)](#) — parameters ( `environment` , `reference` ), `skip` , `environmentOverrides` / `groupOverrides`
- [Monaco manifest and resources \(DT docs\)](#) — "the name value is the name of the environment variable that holds the secret, not the secret itself."
- [Install Monaco \(DT docs\)](#)
- [Monaco Documentation](#)

- [Monaco Examples](#)
- [Configuration Schema Reference](#)

---

## Summary

---

In this notebook, you learned:

- How to install and configure Monaco
- Project structure and manifest configuration
- Creating and deploying configurations via YAML
- Advanced features: reference parameters, `skip`, environment/group overrides, and project-scoped deploys

**Key Takeaway:** Monaco is ideal for teams wanting GitOps-style configuration management. It provides the right balance of simplicity and power for most Dynatrace automation needs. For the from-zero-to-pipeline sequence, see **AUTOM-01 §6 Path B**.

---

*Continue to AUTOM-04: Terraform Provider to learn infrastructure-as-code patterns, or jump to AUTOM-07: CI/CD Integration to wire Monaco into a pipeline now.*

---

## Community Resources & Examples

---

The following GitHub repositories provide starter templates, real-world examples, and hands-on exercises for Monaco:

### Official Repositories

| Repository                                      | Description                                                                                                   |
|-------------------------------------------------|---------------------------------------------------------------------------------------------------------------|
| <a href="#">dynatrace-configuration-as-code</a> | Official Monaco CLI (v2.29.1 at time of writing, 09/2026 — check releases for newer) -- Go binary, Apache-2.0 |

| Repository                                              | Description                                                                                                      |
|---------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| <a href="#">dynatrace-configuration-as-code-samples</a> | Official samples repo with 9 Monaco starter templates in <code>basic-templates-monaco</code>                     |
| <a href="#">easytrade</a>                               | Demo microservices app with a working <code>monaco/</code> directory (manifest.yaml, detection rules, workflows) |
| <a href="#">Dynatrace-Config-Manager</a>                | GUI tool for tenant-to-tenant config migration; complements Monaco for brownfield scenarios                      |

## Starter Templates (in `dynatrace-configuration-as-code-samples` )

| Template Directory                       | What It Configures                                                                                       |
|------------------------------------------|----------------------------------------------------------------------------------------------------------|
| <code>basic-templates-monaco</code>      | Alerting, app detection, synthetic, maintenance window, management zones, ownership, notifications, SLOs |
| <code>learn-monaco-auto-tag</code>       | Auto-tagging with Monaco                                                                                 |
| <code>account-monaco-admin-access</code> | Admin access setup for Monaco                                                                            |

## Pipeline Observability Samples

Monaco configurations for ingesting CI/CD pipeline events via OpenPipeline:

| Directory                                  | CI/CD Platform |
|--------------------------------------------|----------------|
| <code>github_pipeline_observability</code> | GitHub Actions |
| <code>gitlab_pipeline_observability</code> | GitLab CI      |
| <code>azure_devops_observability</code>    | Azure DevOps   |
| <code>argocd_observability</code>          | ArgoCD         |

## Training & Exercises

| Repository                                  | Description                                                                                     |
|---------------------------------------------|-------------------------------------------------------------------------------------------------|
| <a href="#">monaco-self-paced-exercises</a> | 6 structured exercises: install, auto-tag, download, variables, delete/restore, linking configs |
| <a href="#">monaco-demo</a>                 | Working GitHub Actions workflow for Monaco deploy with "crawl-walk-run" adoption methodology    |

**Note:** Monaco v1 ( [dynatrace-oss/dynatrace-monitoring-as-code](#) ) is no longer supported. Current Monaco releases have no conversion command: *"The last Monaco version to support the convert command is Monaco version 2.19.0."* Either run Monaco 2.19.0's `convert` per the [migrating to v2 guide \(DT docs\)](#), or re-run `monaco download` against the source tenant.

---

*This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.*